SEASUG Paper 51-2026

Understanding PROC SQL Execution Order and Its Impact on Query Results

Kiran Venna

ABSTRACT

Understanding PROC SQL execution order is important because PROC SQL statements are not executed in the same order in which they are written. This paper discusses how PROC SQL execution order impacts query results as well as errors encountered while running queries. In this paper, we discuss three scenarios: (1) why certain variables used in earlier clauses are not available to subsequent clauses, (2) differences between filtering using WHERE and HAVING clauses, and (3) how using a WHERE clause instead of AND conditions in LEFT JOIN operations can affect the final output and lead to unexpected results.

INTRODUCTION

When querying using PROC SQL, we use various clauses, and these are used in a specific order:

1. SELECT
2. FROM
3. JOIN ON
4. WHERE
5. GROUP BY
6. HAVING
7. ORDER BY

Most of the time, we may not use all these clauses, but when we do, this is the general order in which they are written. However, when executed in real time, the PROC SQL optimizer defines a different execution order:

1. FROM
2. JOIN ON
3. WHERE
4. GROUP BY
5. HAVING
6. SELECT
7. ORDER BY

PROC SQL's execution order differs slightly from that of ANSI (standard) SQL and includes certain nuances. These differences are beyond the scope of this paper and will not be discussed here.
An improper understanding of how PROC SQL execution works can impact results in unanticipated ways or even cause the code to throw syntax errors.

With this introduction, let us now discuss three scenarios where a lack of execution-order knowledge causes issues.

Scenario One: Why Certain Variables Used in Earlier Clauses Are Not Available to Subsequent Clauses.

Let us understand this concept using the SASHELP.CLASS dataset. We will look at two examples. In the first example, we will use a variable that already exists in the dataset. In the second example, we will create a new variable in the SELECT statement and see how the execution order affects its availability in the WHERE clause.

Example 1: Using an Existing Variable

In the first example shown below, we use the AGE variable, which already exists in the SASHELP.CLASS dataset.

```
proc sql;
select *
from sashelp.class
where age > 15;
quit;
```

The query executes successfully, and the result contains the observations where AGE is greater than 15, as shown below.

Name	Sex	Age	Height	Weight
Philip	M	16	72	150

This works because AGE is an existing variable in the SASHELP.CLASS dataset. The FROM clause identifies the source table, and the WHERE clause can access the variables that already exist in that table.

Example 2: Using a Newly Created Variable

Now, let us create a new variable in the SELECT statement and try to use that variable in the WHERE clause as shown below.

```
proc sql;
select *, age+100 as newage
from sashelp.class
where newage > 115;
quit;
```

Here, NEWAGE is not an existing variable in the SASHELP.CLASS dataset. It is a new variable created by the SELECT statement.

As discussed earlier, the logical processing of the WHERE clause occurs before the SELECT clause. Therefore, when SAS processes the WHERE clause, the NEWAGE variable has not yet been created.

As a result, the query generates an error, as shown below.

```
ERROR: The following columns were not found in the contributing tables: newage.
```

Circumventing the Problem

However, we can avoid this problem by using the same expression or calculation directly in the WHERE clause as shown below.

```
proc sql;
select *, age+100 as newage
from sashelp.class
where age+100 > 115;
quit;
```

In this case, we are no longer referring to NEWAGE in the WHERE clause. Instead, we are using the expression AGE + 100 directly. Because AGE already exists in the source dataset, the WHERE clause can evaluate the expression before the SELECT clause creates the NEWAGE variable.

The results are shown below.

Name	Sex	Age	Height	Weight	newage
Philip	M	16	72	150	116

This example demonstrates why understanding the logical execution order of SQL clauses is important. When a newly created variable is not available to a subsequent

clause, we can sometimes circumvent the problem by using the underlying SQL expression or calculation directly in that clause.

Understanding the execution order not only helps us avoid syntax errors but also helps us write SQL queries that produce the expected results.

Scenario two: Differences Between filtering using WHERE and HAVING clauses.

In this section, we'll examine how filtering data with WHERE versus HAVING clauses impacts query performance, and how this relates to SQL's logical execution order and data processing.

To illustrate this clearly, let's first create a large dataset so the performance impact of filtering is easier to observe. We'll do this by duplicating each row of SASHELP.CARS 100,000 times, producing a dataset of roughly 42.8 million records. Here's the code:

```
data work.cars_large;
      set sashelp.cars;
      /* Duplicate every single row 100000 times */
      do i = 1 to 100000;
            output;
      end;
      drop i;
run;
```

Filtering with WHERE:

Now let's filter the data using a WHERE clause, restricting results to Make = 'Acura' while also calculating the average MSRP by Make using the AVG aggregate function:

```
proc sql;
select make,
 avg(msrp) as avg
from cars_large
 where Make ='Acura'
group by Make ;
quit;
```

Runtime and memory usage:

```
NOTE: PROCEDURE SQL used (Total process time):
      real time 2.03 seconds
      user cpu time 1.19 seconds
```

```
      system cpu time 0.99 seconds
       memory 40700.32k
      OS Memory 62584.00k
```

Filtering with HAVING:

Next, let's perform the same filter, but this time using a HAVING clause instead of WHERE:

```
proc sql;
select make,
 avg(msrp) as avg
from cars_large
group by Make
having Make ='Acura';
quit;
```

Runtime and memory usage:

```
NOTE: PROCEDURE SQL used (Total process time):
      real time 6.14 seconds
      user cpu time 12.84 seconds
      system cpu time 2.34 seconds
      memory 2224246.07k
     OS Memory 2246324.00k
```

Comparing the Results:

Comparing the two runs, the HAVING clause query took roughly 3 times longer in real time and consumed roughly 54 times more memory than the WHERE clause query.

The reason lies in when filtering occurs relative to aggregation. With WHERE, filtering happens before the data is grouped and aggregated, so SAS only has to aggregate the small subset of rows where Make = 'Acura'. With HAVING, the filter is applied after grouping and aggregation, meaning SAS must first group and aggregate the entire 42.8 million row dataset by Make, and only afterward discard all groups except Acura.

Even though both queries specify the same filter condition, the point in the logical execution order at which that filter is applied makes a significant difference. Understanding this distinction, and SQL's underlying execution order in general, is key to writing efficient queries and optimizing performance, especially when working with

large datasets where avoiding unnecessary processing can meaningfully reduce runtime.

Scenario Three: How Using a WHERE Clause Instead of AND Conditions in LEFT JOIN Operations Can Affect the Final Output and Lead to Unexpected Results

Although the execution order isn't always explicitly stated, an AND condition in a join works very differently from a WHERE clause. In a LEFT JOIN, the WHERE clause is applied only after the join is complete, while an AND condition is applied as part of the join itself. This difference can dramatically impact the results. Let's look at some sample data to understand this.

To understand this scenario, it's important to first review what happens in a typical LEFT JOIN. In a LEFT JOIN, we retrieve all rows from the left (or first) dataset mentioned in the join, along with matching rows from the right table. First, let's look at the sample datasets:

```
/*left dataset*/
data one;
input id name $;
datalines;
1 Smith
2 John
3 Sam
;
run;
data two;
input id age;
datalines;
1 30
2 40
4 45
;
run;
```

Now let's use the following code for the LEFT JOIN:

```
proc sql;
select a.*
 ,age
from one a
left join two b
on a.id=b.id;
quit;
```

From the code above, we retrieve all variables from dataset one along with the age variable from dataset two. Whenever there's a match between the joining columns, the corresponding age from dataset two is included; otherwise, it will be missing, as shown in the result below.

id	name	age
1	Smith	30
2	John	40
3	Sam	.

Adding an AND Condition:

Now let's see what happens when we add an additional join condition using AND. In the LEFT JOIN code above, we add the condition age = 40:

```
proc sql;
select a.*
,b.age
from one a
left join two b
on a.id=b.id
and age=40;
quit;
```

This extends the typical behavior of a LEFT JOIN: we retrieve everything from the left table, and values from the right table are joined only when both the id matches and age equals 40. All other rows still appear, but their age values are null, as shown below.

id	name	age
1	Smith	.
2	John	40
3	Sam	.

Replacing AND with WHERE:

Now, if we replace the AND condition with a WHERE clause instead:

```
proc sql;
select a.*
,b.age
from one a
```

```
left join two b
on a.id=b.id
where age=40;
quit;
```

Modifying the AND condition with a WHERE clause can yield entirely different results. In this case, we now have only one row instead of all the rows from the LEFT JOIN with the previously mentioned AND condition as shown below.

id	name	age
2	John	40

Why This Happens

In a LEFT JOIN, we normally expect to see all rows from the left table. However, when a WHERE clause is applied, it filters the results of the join rather than becoming part of it. This happens in two steps:

1. **Joining:** The tables are joined based on the condition specified in the ON clause, producing a temporary result set that includes all rows from the left table, with matching values (or nulls) from the right table.
2. **Filtering:** The WHERE clause is then applied to this temporary result set, removing any rows that don't meet the specified condition.

This sequence shows how the WHERE clause operates after the join is complete, which directly influences which rows make it into the final result.

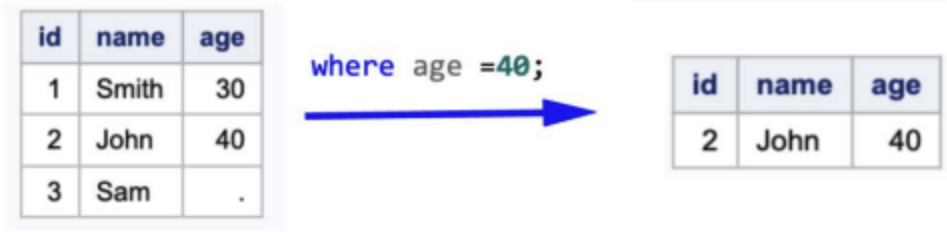
In step one, the join condition is applied, producing the results shown below.

```
from one a
left join two b
on a.id=b.id
```



id	name	age
1	Smith	30
2	John	40
3	Sam	.

In step two, the results from the previous step are filtered using the WHERE clause. Since this filtering happens after the join, and the WHERE clause discards any row where age isn't 40 (including the null values produced for non-matching rows), only one row remains in the final output.



From the above discussion, we need to be careful when using the AND condition and the WHERE clause, choosing one over the other based on specific business requirements.

Conclusion

Finally, it is important to remember that we need to understand how PROC SQL queries are executed. We have seen that if we are not aware of how newly created variables behave in a WHERE clause, we can encounter syntax errors. Improper filtering of data will impact performance, so we should always filter data as early as possible, as this gives better efficiency since we process less data. It is also important to understand how AND conditions and WHERE clauses can impact results, given the way they are executed.

Contact Information

Your comments and questions are valued and encouraged. Contact the author at:

Kiran Venna

Email: kvenna.sas@gmail.com

SAS and all other SAS Institute Inc. product or service names are registered trademarks or trademarks of SAS Institute Inc. in the USA and other countries. ®indicates USA registration.